Problem Statement and Goals
Lake

Team #10, CinqGarçons
Ethan Walsh
Colin Chambachan
Matthew Nesbitt
Jinwoo Hong
Gary Qin

Table 1: Revision History

Date	Developer(s)	Change
September 28, 2026	Ethan Walsh	Sections 1, 2, 4 and 5 (Problem Statement, Goals, Custom Documents, Generative AI Use Disclosure)
September 28, 2026	Jinwoo Hong	Section 3 (Stretch Goals)
September 28, 2026	Gary Qin	Reflection appendix
October 4, 2026	Ethan Walsh	Refinement of Inputs/Outputs and addition of impact statement to end of Problem Statement
October 15, 2026	Gary Qin	Clarified which privacy protections are in the core scope (Sections 1.1 and 3.4)

1 Problem Statement

1.1 Problem

As the use of artificial intelligence becomes more common, the most popular option is to use a commercial large language model from a major inference provider via a web interface or API (application programming interface). This approach is subject to variable costs and insufficient data privacy protections. Although some users have opted for open-source models on local devices, these models are often limited in quality due to hardware restrictions. Users currently

cannot access the benefits of both approaches: low-cost, high-quality, private artificial intelligence. Addressing this gap would create more democratized access to large language models.

This project aims to build a system to partition and distribute a given artificial intelligence model across a local network of devices, enabling improved performance and model capacity while operating on private, self-owned infrastructure.

In the core project, privacy means that prompts, responses and model data stay on devices within the user's own local network and are never sent to an external AI provider. The core project does not encrypt traffic between devices or authenticate devices before they join (see Section 3.4). Users must therefore run Lake only on devices they trust, connected by a local network they control, since other devices on that network could read the traffic between devices.

1.2 Inputs and Outputs

Inputs:

- User inputs to the interface (clicks and button presses)
- User's prompt to the distributed model
- Artificial Intelligence Model (large language model using transformer architecture)

Note that at this stage of the project, the model is considered an input since the project is focused on splitting up the model. The user chooses the model and the system will partition it depending on the device pool.

Outputs:

- User Interface
- Model response(s) to user prompt
- Partitioned model sections (split up based on device pool)

1.3 Stakeholders

Stakeholders include, but are not limited to:

- Local AI users who run models on their own device(s)
- Small companies in regulated industries (more specifically, industries where data privacy is paramount)
- Low-budget and smaller AI labs focusing on research related to AI models

An example of a small company in a regulated industry is a local medical clinic. This clinic may require AI or computer vision based analysis of patient scan results but this data must remain private.

1.4 Environment

1.4.1 Hardware

Hardware used for this project includes consumer devices such as personal computers and laptops. Mobile phones may also be used in the final iteration of the project but is not in-scope for the initial proof of concept. Hardware compatability will be limited to devices containing GPUs (graphics processing units) manufactured by Nvidia, AMD, or Apple. Any hardware requirements beyond this will be specified and listed as the project moves further into the development phase.

1.4.2 Software

Software must be compatible with various operating systems to be compatible on consumer devices. This includes Windows, Linux, and MacOS.

2 Goals

2.1 Improved Local Performance Outcomes

The system's model will achieve better performance than if the equivalent model is only run on one device. The aim of this goal is to generally improve local AI outcomes and performance so that the product can become a viable alternative to commercial LLMs for simple tasks. "Better" will be measured in a variety of metrics:

- Primary Metrics:
  - Model size that can be accommodated (number of parameters and quantization)
- Secondary Metrics:
  - Response time
  - Throughput (tokens per second)
  - Model benchmarking

This goal will be considered complete if the primary metric is substantially improved. Improving the secondary metrics will be considered as additional bonus.

2.2 Model Parallelization

This goal is the motivating question that brought about this project: can a model be parallelized across multiple local devices and run successfully by producing coherent output? This goal is defined as the system parallelizing an

artificial intelligence model (more specifically, a large language model based on the transformer architecture) across more than one local device. Currently, this goal will be considered met when the system is able to split up the layers of a model across the pool of available devices.

2.3 Cross-Platform Optimization

The system is able to operate across different operating systems and platforms (e.g. Windows, Mac, and Linux) so that users may easily connect their devices. Achieving this will result in the user base not being limited based on their existing device type.

2.4 Dynamic Allocation

In the case goal 2.2 can be achieved (model parallelization), the system can dynamically allocate the parallelized model across the pool of available devices. This can be done when a device joins or leaves the network, given that there are sufficient pooled resources to run the model. This is a key shortcoming of existing local distributed inference providers that only do one allocation pass at the start of the session. By achieving this goal, the system would be more efficient and resilient since it can continue to operate as devices drop or are added to the network. The success of the goal will be evaluated by verifying that the distributed model can serve requests after model layers are reassigned, as well as measuring the reallocation time.

3 Stretch Goals

3.1 Wired and Wireless Connections

The system can operate via both wired and wireless connection (v0: wired, v1+: wireless). The network latency and variability imposed by a wireless connection may make the distributed system throughput too low for practical use.

3.2 Extended Cross-Platform Support for Smartphones

iOS and Android inference is commonly supported by inference engines, but practical feasibility of distributing inference to smartphones remains unknown, due to the limitations of smartphone hardware.

3.3 Concurrent Inference

The distributed system can serve inference to at least two concurrent users at the same time, rather than completing singular requests sequentially. Each request must keep its own state so user outputs do not interfere with one another. This would improve aggregate throughput and observed latency when multiple users are using the model.

3.4 Privacy and Security

Communication between devices is encrypted, protecting requests and intermediate data from other devices that may be observing the network. Devices can also authenticate before joining the network cluster, protecting data in transit and ensuring only trusted devices are utilized. This goal removes the need for the trusted-network assumption in Section 1.1, making Lake suitable for networks that untrusted devices also use.

4 Custom Documents (CDs)

- **Fall:** Design Thinking Report
- **Winter:** Performance Report

5 Generative AI Use Disclosure

Anthropic's Claude Sonnet 5 was used as a method to provide feedback on early revisions of the problem statement (rubric and example problem statements from the capstone GitLab were provided as context). No text generated by the model was used and only general suggestions were used as guidelines for improvement. Google's AI (built into search functionality) was also used as a method to provide feedback on the environment section (however feedback was largely ignored and deemed to be unhelpful).

Appendix — Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. How did you and your team adjust the scope of your goals to ensure they are suitable for a Capstone project (not overly ambitious but also of appropriate complexity for a senior design project)?

---

1. This deliverable went smoothly, and a key reason was because our team has done a lot of discussion about what we wanted our Capstone experience to be like prior to the start of the term. Many of our prior conversations involved someone bringing up a potential idea for the project, followed by the rest of us questioning its details to try to understand it completely. That's why we've found that, when it came time to start putting down documentation for these deliverables, the answers came quickly because we've already discussed many of these topics.
2. Despite our regular discussions on project decisions, we still found that there were aspects of the project we hadn't yet considered but disagreed on. For example, when it came to choosing our desired Git workflow, some members wanted to use a feature branch workflow while others wanted to use a forking workflow. Both sides felt pretty strongly about their choice, so we debated at length on which workflow was better. We eventually decided upon a middle ground by using both workflows after an impassioned discussion, and we made sure to resolve things nicely by reminding each other of a few things: 1, nothing is personal when we're working through project stuff, and 2, working through & talking about disagreements is a better option than brushing things under the rug.

3. Scope was an important aspect for our team to consider because for many of us, this was our first foray into working with technologies surrounding LLMs. Due to this, we initially didn't have a great idea on what would be a feasible project for Capstone. To educate ourselves, we looked into related projects and research papers to get a grasp on the main concepts behind our distributed LLM idea. Once we understood the core concepts behind the technology and learned about the new ideas that related projects were bringing to the table, we broke down the project into manageable sections: the core functionality needed for a minimum viable product, and extensions to the core that make the project more interesting. This ensures that the scope of the project is not overly ambitious because the core is minimal, but it also ensures appropriate complexity because we have many potential extensions that could add more functionality, compatibility, or usability to the final project.